Primitive recursion is computable via the beta function and regular minimization

The Open Logic Project contributors

openlogic:incompleteness/representability-in-q:local-snapshot-20261013; CC BY 4.0

Editorial note. All delivered mathematical text is the author's own, copied from the pinned Open Logic Project unit file sources/olp-incompleteness-representability-in-q/pinned-original-source.tex; no statement or proof step was rewritten, reordered or removed.

Editorial note. The counted proof is the author's complete argument: the author's parenthetical 'no primitive recursion is needed here, just minimization' is retained verbatim

Editorial note. The label lem:prim-rec names the counted statement, and the proof environment immediately following it in the source is delivered verbatim as its proof.

Editorial note. The counted statement is carried by the author's own lem environment in the unit source.

Allowed standard prerequisites (editorial).

- The beta function lemma. Same unit, \ollabel{lem:beta} (beta-function section). Regularity of the minimised predicate: for every prefix of the recursion there is a sequence code d realising it, so the minimum exists.

Source reference. Original lemma lem-beta: sources/olp-incompleteness-representability-in-q/pinned-original-source.tex, source lines 656–656. Prerequisite pointer invoked at proof line(s) 1614; it names a standard fact of the same unit that the counted argument uses, and every step of the counted proof built on it is retained verbatim in the delivered body.

Lemma 0.1. *If h can be defined from f and g using primitive recursion, it can be defined from f, g, the functions* 0, S, projni, Add, Mult, $\chi_=$, *using composition and regular minimization.*

Proof. First, define an auxiliary function $\hat{h}(\vec{x}, y)$ which returns the least number d such that d codes a sequence which satisfies

1. $(d)_0 = f(\vec{x})$, and
2. for each $i < y$, $(d)_{i+1} = g(\vec{x}, i, (d)_i)$,

where now $(d)_i$ is short for $\beta(d, i)$. In other words, $\hat{h}$ returns the sequence $\langle h(\vec{x}, 0), h(\vec{x}, 1), \ldots, h(\vec{x}, y)\rangle$. We can write $\hat{h}$ as

$$\hat{h}(\vec{x}, y) = \mu d(\beta(d, 0) = f(\vec{x}) \wedge \forall i < y\beta(d, i + 1) = g(\vec{x}, i, \beta(d, i)).$$

Note: no primitive recursion is needed here, just minimization. The function we minimize is regular because of the beta function lemma lem-beta.

But now we have

$$h(\vec{x}, y) = \beta(\hat{h}(\vec{x}, y), y),$$

so h can be defined from the basic functions using just composition and regular minimization. □